



This dice is so unpredictable!
How does it work?

Simple coding is used to make the
dice show a random number on every
roll. Let me show you how it's done.



Dice are used in board games like Monopoly, Picnic, Ludo, Snakes and Ladders, and many more! A dice is a cube-shaped 3D (length, width and height) object. It has 6 sides which are called faces. Each face has dots between 1 and 6. We cannot predict the outcome of a dice before it is rolled.



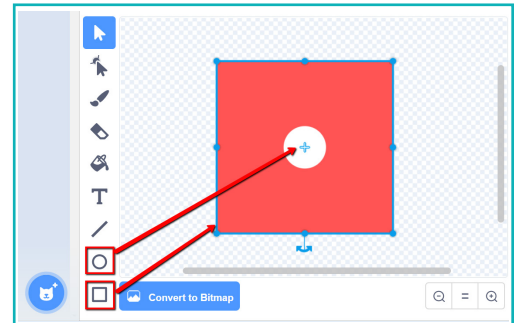
Dice are an essential part of board games. However, we can use Scratch to create dice on computers and mobile phones, so we can use them anywhere.

1. Create a dice in Scratch to make a board game:

We will learn to create a 2D dice which shows the top face. A 2D dice will have only 2 dimensions: length and width.

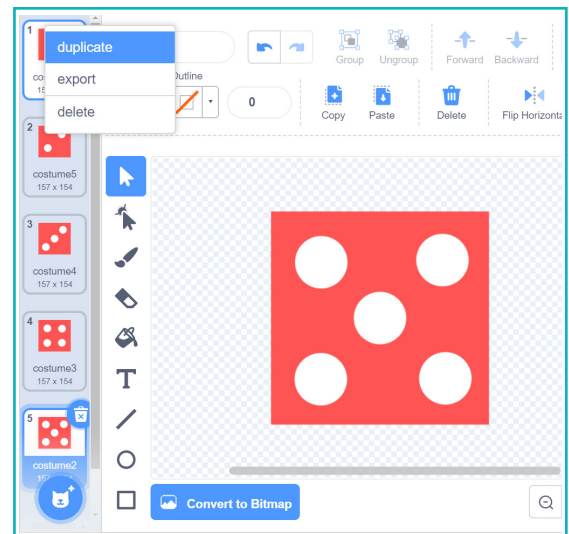
We need to create each face of the dice:

- In 'Choose a Sprite', use the 'paint' option to create a 'dice' sprite.
- Create a dice face using the rectangular and circular drawing tools with appropriate colours.



Note: Match the centre of the face with the centre mark on the costume canvas.

- Duplicate the costume 5 times to create each face. There will be 6 in total.
- Duplicate the white circles and edit each costume to match the faces of the dice.



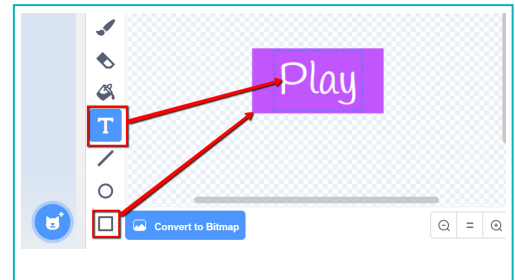
- Confirm that number of dots on the costumes and costume number are same.



Note: Arrange the costumes in a sequence so that costume 1 is first and costume 6 is sixth. This will make sure the current costume is shown on stage.

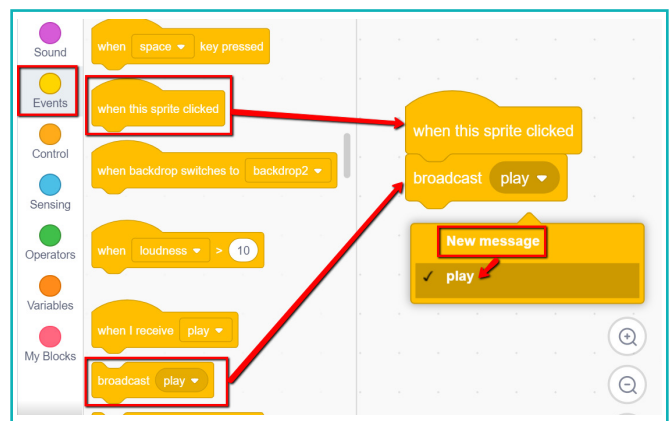
2. To roll the dice, we need to create a play button sprite

a. In 'Choose a Sprite', select the 'paint' option to create a blank sprite. Name it 'Play'.



b. In the 'Play' sprite, add the 'when sprite clicked' block. Add the 'broadcast message' block for the Play button to run.

c. Create a new broadcast message called "play".

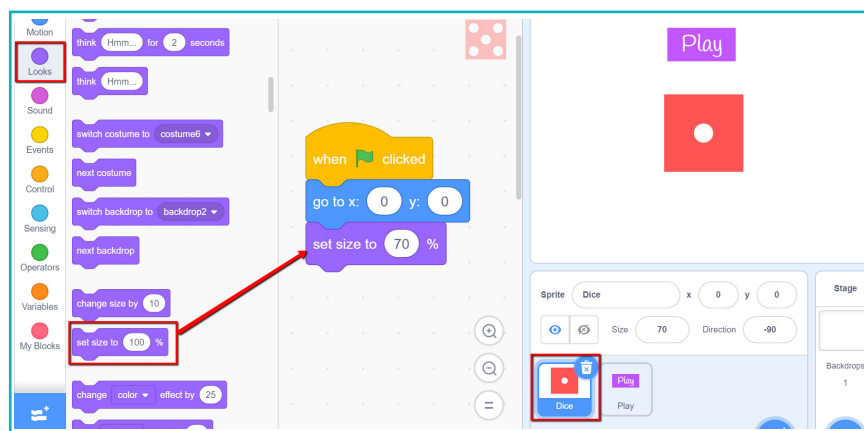


d. Create a new broadcast message called "play". When Play is clicked, the dice should roll. Let's create a code to position the dice sprite on the stage.

e. To position the dice at a fixed place when  is clicked, add the  block.

f. Add 'go to x y' and position the dice at the centre of the stage for now.

g. Add the 'Set size' block to set the initial size of sprite at around '70%'. This will make sure the dice is not too big.



When we roll a dice, it rolls over a few times and settles on a number. It gives a random outcome on its top face. We use the number on its top face to play. We can use the operators to create this in Scratch.



Recall:

Operators: An operator block is a light-green coloured block used to perform comparisons, math calculations, and modifications of words or sentences.

We learnt about comparison operator blocks in the previous lesson. In this lesson, we will learn about other operators.

- Pick random block.



This block picks a random number between the two given numbers, including both numbers.

- For example:



This block will randomly pick any one number from the range of numbers 1,2,3,4,5,6 when clicked or run. There is no bias in the result.

- **Calculation operator blocks:** These blocks are used to perform calculations like addition, subtraction, multiplication, and division.

- For example:



This block performs addition. Here 2 and 3 are added to give the result 5.



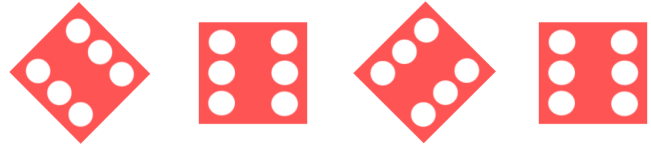
This block performs subtraction. Here 3 is subtracted from 10 to give the result 7.



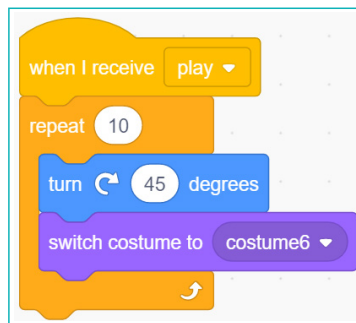
Variables can be used in these blocks. For example, this block will subtract the value in the y variable from the value in the x variable. When $x=5$ and $y=3$, this block will give you the answer 2.

3. Add a 2D rolling action to the dice

We can rotate the sprite multiple times by 45 degrees.



- Add the 'When I receive broadcast' block from the 'Events' palette to receive the 'play' message.
- Add a 'repeat' block to the 'when I receive broadcast' block. This will rotate the dice multiple times.
- Insert the 'turn' block and set to rotate by '45' degrees. Switch the 'costume' block to change the number on the top face. Since we have created different costumes for each face of the dice.

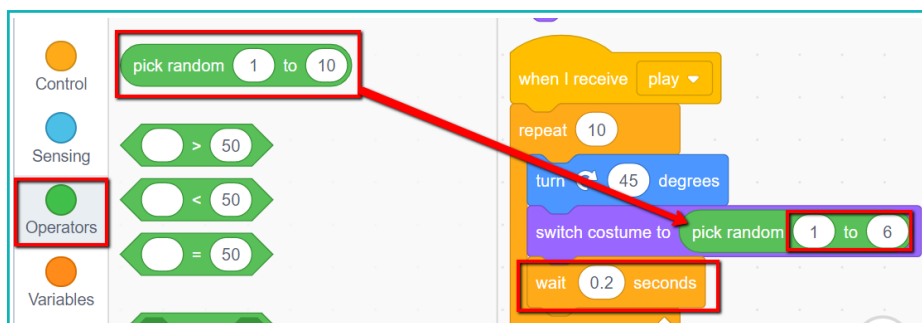


- We need the dice face to change randomly. We need to switch the costume to randomly choose one of the six faces for a dice roll.

4. Set the dice to display a random number on the top face

We will use the 'Pick random' block from operators.

- To get a random number between 1 to 6, use the 'pick random' block. Add it as input in the 'switch costume to' block.
- Add the 'wait' block and set it to '0.2' seconds. This will make the dice's movement seem natural and not too fast.



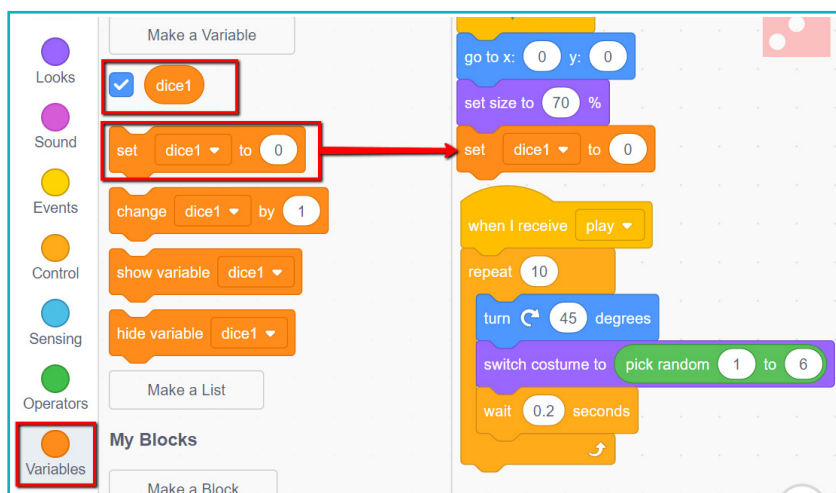
- Click on the play button to watch the dice roll. The number is displayed on the dice face.

5. Display the number on the top face using digits

Numbers can be saved in variables and the value displayed on the stage. We can use variables to display the number.

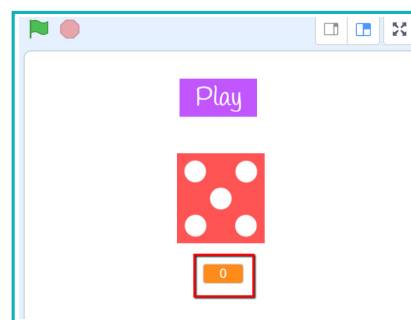
Can you recall variables from the last Scratch session?

- Create a variable named “dice1”.
- Set the initial value of the variable to ‘0’, as it will have no value before the dice is rolled.

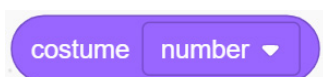


- The value in the variable is displayed on the stage if tick marked in the ‘Variables’ palette. This value is displayed in 3 forms. You can double click it to change the form.

- Place the variable’s second form below the dice on the stage. This will display the number on the dice.

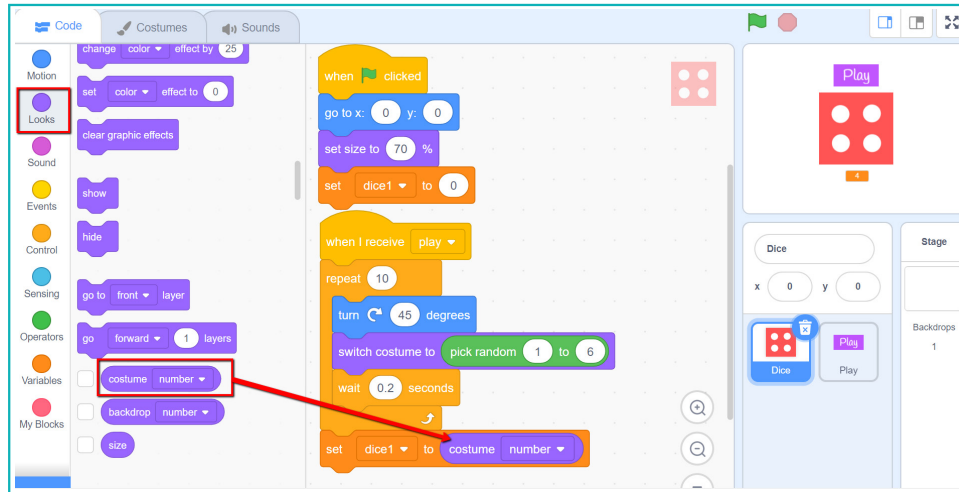


Since dice costumes are arranged sequentially, the costume number is same as the dice number. Therefore, the current costume number will be the number on the top face of the dice.



This block contains the sequence number of the costume which is currently visible on the Stage.

e. To display the number on the face of the dice, set the variable to the current costume number.



f. Now when the dice rolls, we can also view the variable number below the dice.

Exercise

State whether True or False

1 **costume number** is a default variable block in Scratch that stores the value of sequential number of current costumes shown on stage.

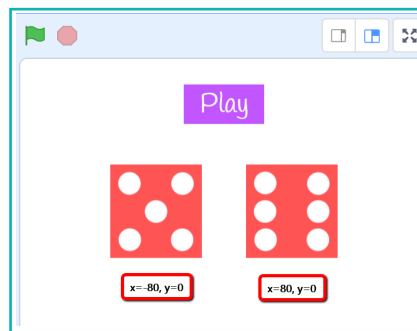
2 **pick random 2 to 5** block will provide any random value between 2 to 5.

Most games require 2 dice to play.

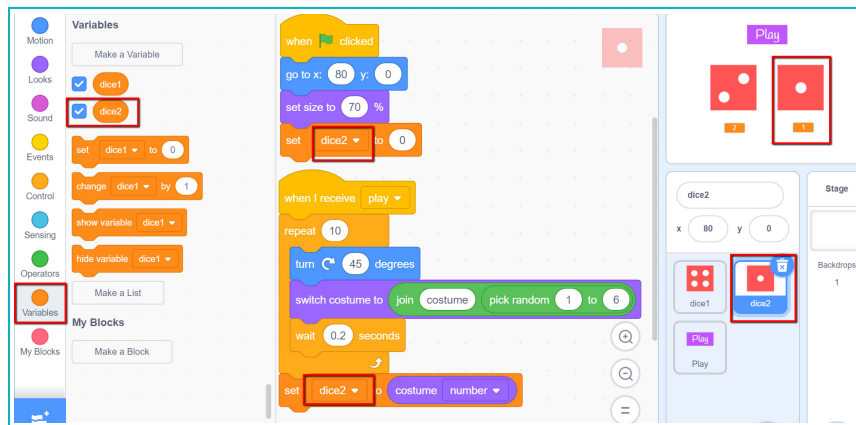
6. Change the dice sprite name from 'dice' to 'dice1'.

- Duplicate the current dice to add a second dice. Name it 'dice2'.
- Place the dice side by side and update their initial position in the 'go to x y' block.

- For **dice1**, **x = -80, y = 0**.
For **dice2**, **x = 80, y = 0**.



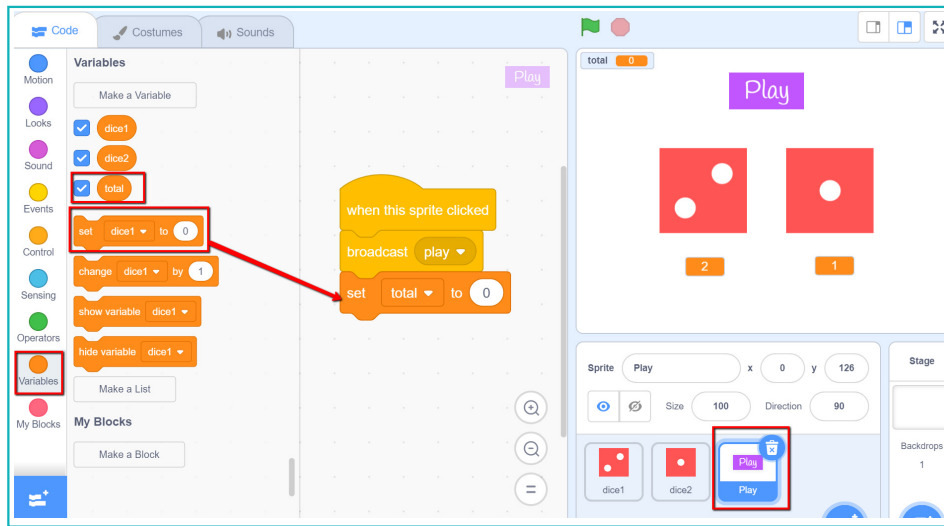
- Create another variable named 'dice2'.
- To record the count on dice 2, update the variable command in the 'dice' sprite from 'dice 1' to 'dice 2' in the dropdown.



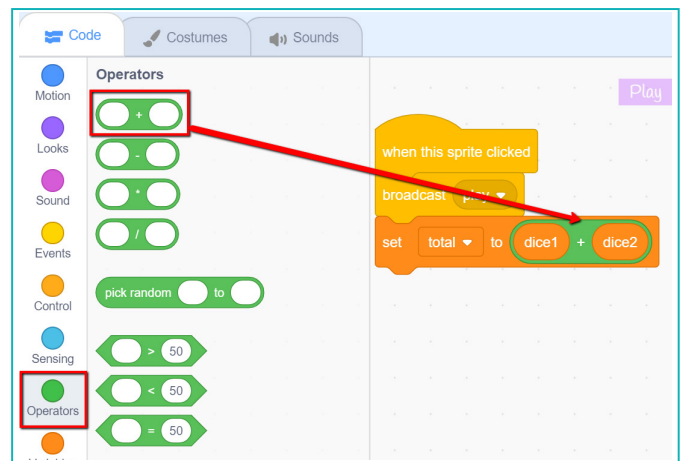
Now we have two rolling dice with their values displayed below each one.

7. Display the total result when numbers from both dice are added.

- To display the total of the numbers on both dice in the 'Play' sprite, create a new variable 'total'.
- Add the 'set variable to' block to set it to equal the addition of variables in dice1 and dice2.



- c. To add values in variables dice1 and dice2, add the 'addition' operator block.



If we click play, the correct total will not be displayed in the 'total' variable. This is because the block broadcasts a message to roll the dice, but doesn't wait for the result. It immediately runs the next command to add variables in dice 1 and 2 before the dice has finished rolling.

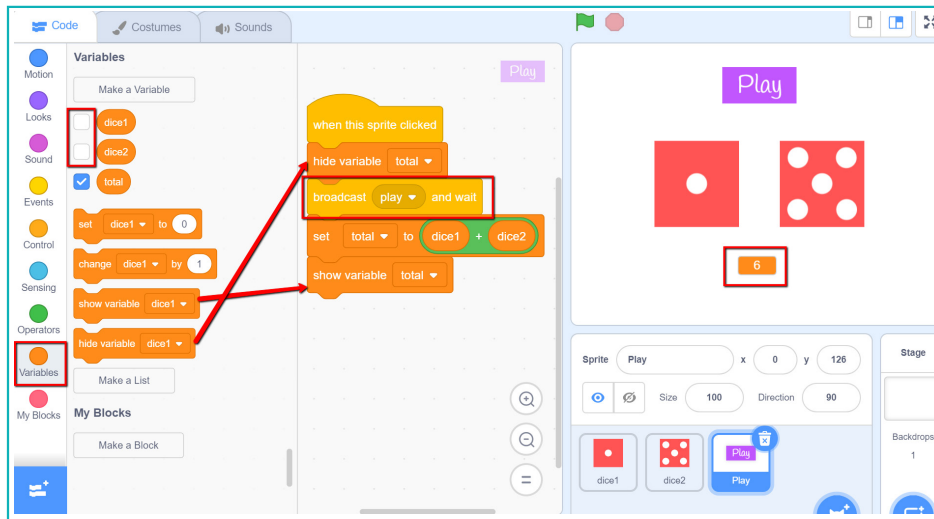


This block also sends a broadcast throughout the Scratch project. It waits until all scripts are activated by the broadcast end and stop running.

When we use the 'broadcast message and wait' block, values in the dice1 and dice2 variables will be added only after dice has finished rolling. This will add together the final values on the dice and display the correct total.

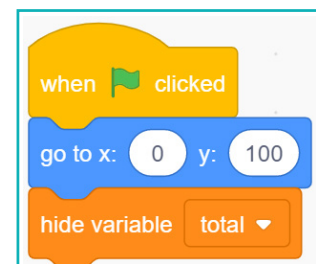
- d. Remove the 'broadcast message' block and add the 'broadcast and wait' block instead.
- e. Uncheck dice1 and 2 in the 'variable' palette. We don't want to display the individual counts on the stage, we only want the total.

- f. Hide the total on the stage when the dice is rolling. It should only be visible afterwards. Use the 'hide and show' variable blocks to do this.



- g. Add code for the initial setup of the Play sprite when the green flag is clicked.

- h. Now, click the green flag to run the code and play the dice game.



Exercise

☒ Tick mark ☒ the correct answer

- 3 Which of the following blocks is used for multiplication?

a. ☐



b. ☐



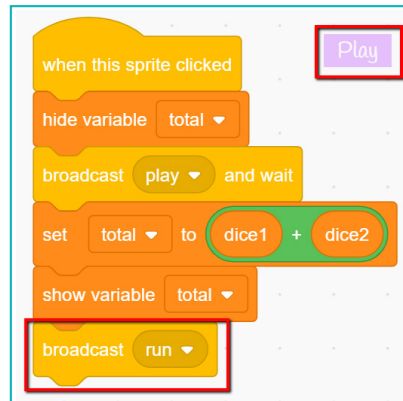
c. ☐



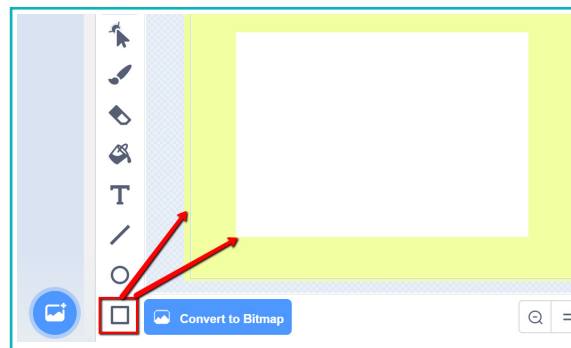
In most dice games, a player moves their piece as many steps as the dice displays. Let's add a sprite that will move as per the output of the rolled dice. This will help to create a complete board game in Scratch.

When the value in total variable is updated, we can use the total to move a character.

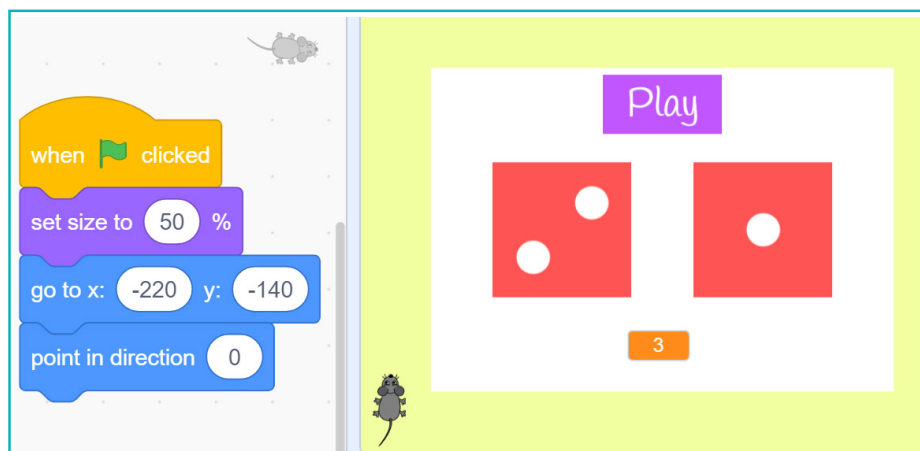
- a. Set the value in the total variable and create a new broadcast 'run' in the 'Play' sprite.



- b. Use the 'square' tool to create a backdrop with a path along the edge.

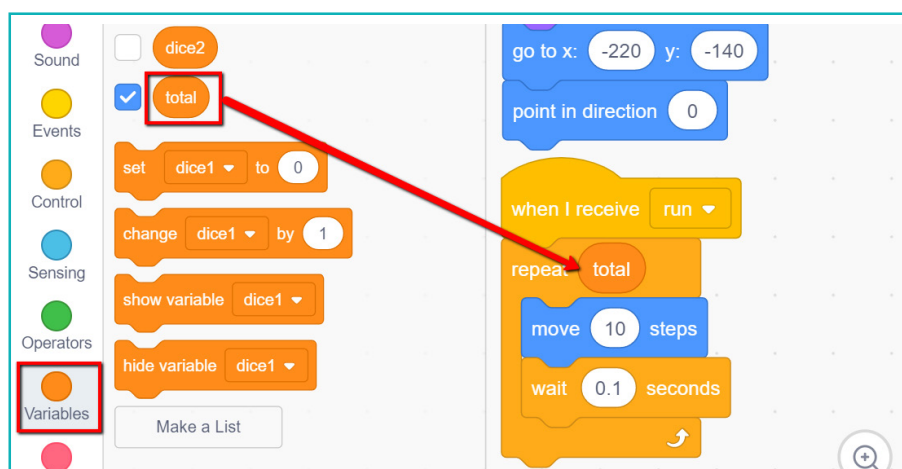


- c. Add a 'mouse' sprite from the library.
d. Set its initial position to the bottom left. This will be its start point when the green flag is clicked.
e. Set its size to '40%'. This will make it suitable for the path.



We can use a variable with the dice value as an input for the 'repeat' block. The 'move' block can be moved inside the repeat block. This way, the number of times the move block will be repeated is equal to the total values of both dice.

- f. Add the 'when I receive run broadcast' block.
- g. Add the 'repeat block' to repeat the code. It will repeat as many times as the value in the total variable.
- h. Add the 'move 10 steps' and 'wait' blocks inside 'repeat'. This will move the sprite one step at a time.

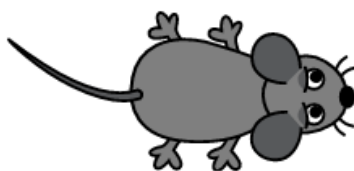


- i. If the mouse reaches the edge, it won't be able to move ahead.

Use the If-Else Condition to make the mouse turn when it reaches the edge.

If-Else condition: This block is used to take decisions. If the condition is true, the first set of blocks runs. If not, the second set runs. Only one of the two sets can run.

For example: Here, the mouse should check the condition 'if touching the edge'.



Condition: If touching the edge?

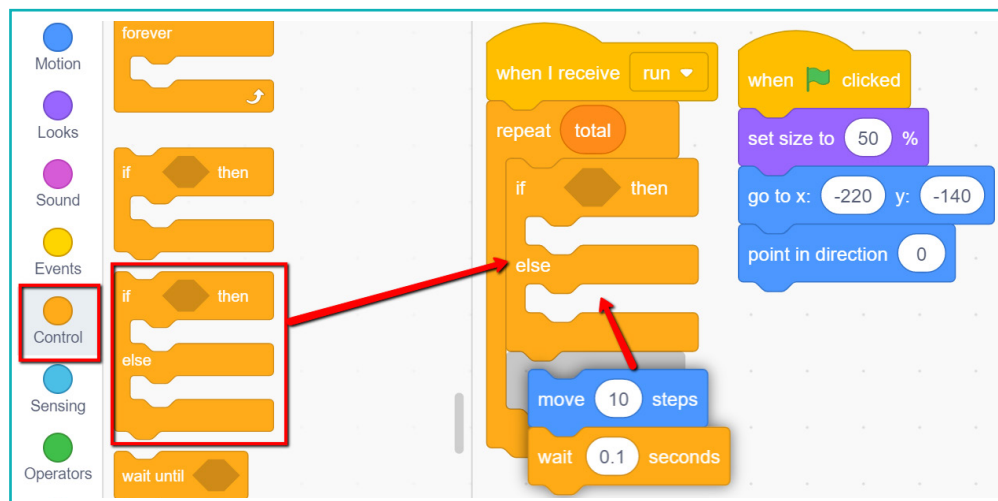


Only one of the two actions will take place depending on the output of the condition.

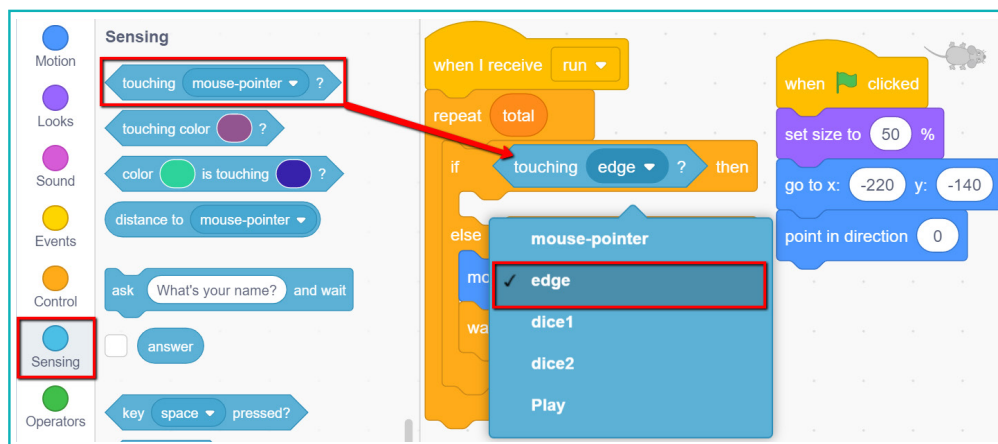
We will use the 'if-else' block inside the repeat block to check if the mouse sprite has touched the edge.

j. Add the 'if else' block inside the repeat block.

k. Under the 'else' block, add the current code to move the sprite ahead.

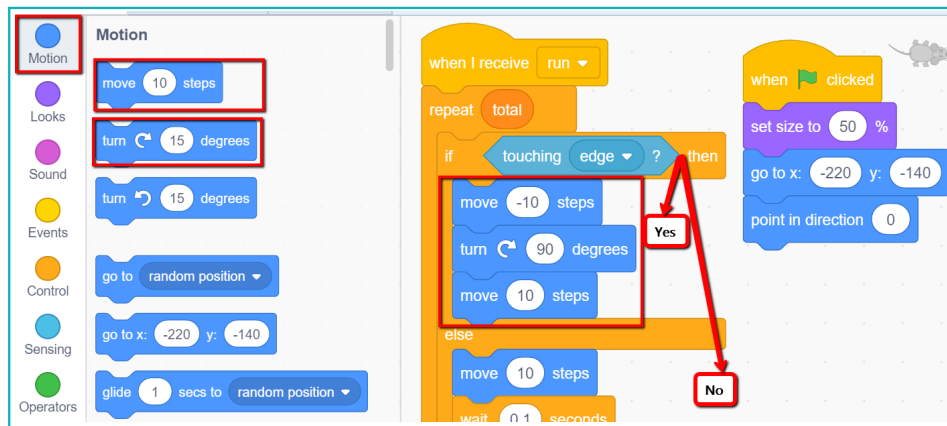


l. Add the 'sensing' block 'If touching edge' as a condition in the 'if else' block.



When the mouse turns at the front edge, its tail touches the side edge. To avoid this and turn smoothly, we will move the mouse back, turn it, and then move it a few steps forward.

- m. We want to avoid the mouse touching the edge. If it touches, the mouse will move 10 steps backwards, turn right and move 10 steps ahead.



- n. Run the animation to watch the mouse move when we roll the dice.

In this lesson, we learnt how to successfully create digital dice using operations. In the game, you can enjoy mouse roaming around with the help of the dice and explore the operation functions to create your own dice board game in the future.

Exercise

State whether True or False

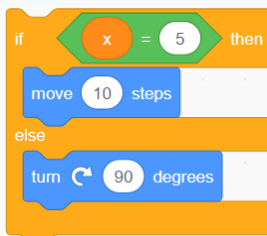
- 4 In the if else block, if the condition is satisfied, then code in the else part is executed.

- 5 To perform a calculation with 3 numbers, one calculation block can be inserted inside another calculation block as shown in the image.



Tick mark ☒ the correct answer

- 6 If the value in variable x is 10, what will be the output on stage when the following code is run?



- a. Sprite will turn right ☐
- b. Sprite will move ahead 10 steps ☐
- c. Sprite will turn right and move ahead 10 steps ☐
- d. Nothing will happen ☐

- 7 Operator blocks are used for:

- | | | | |
|----------------|--------------------------|------------------------------|--------------------------|
| a. Calculation | ☐ | b. To obtain a random number | ☐ |
| c. Comparison | ☐ | d. All of them | ☐ |



Tech Flash

- **Calculation operators** : These blocks are used to perform calculations like addition, subtraction, multiplication and division.
- **Random operator** : This block picks a random number between two given numbers, including both the numbers.
- **If Else condition** : This block is used to take decisions. If the condition is true, the first set of blocks runs. If not, the second set runs. Only either one of the two is run.